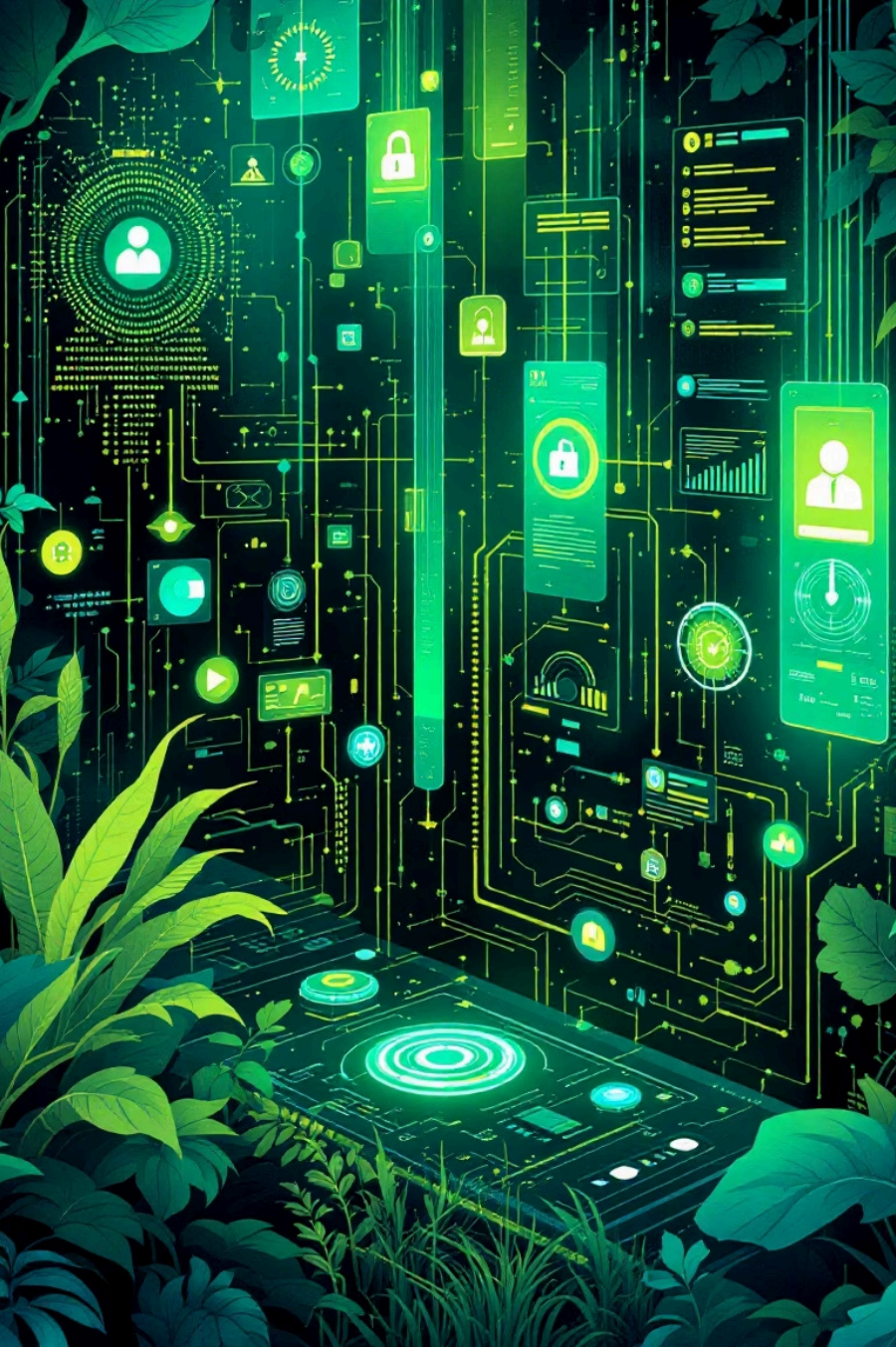




Mobile Incident Response & Threat Hunting

Clase 5: Arquitectura Android, Internos de ADB y Análisis de WhatsApp - Enfoque en Live Triage y Extracción Forense

Objetivos Técnicos de la Sesión



Arquitectura Android

Comprender el modelo de seguridad basado en Sandbox y la estructura del sistema de archivos. Analizar cómo el aislamiento por UID protege los datos de aplicaciones.



Live Triage con ADB

Dominar técnicas de reconocimiento en tiempo real, incluyendo listado de procesos activos, conexiones de red y extracción de artefactos volátiles.



Bypass de Restricciones

Explorar métodos de acceso a la partición /data utilizando privilegios de Root y técnicas alternativas como Android Backup.



App Forensics

Realizar extracción y descifrado de bases de datos SQLite críticas, con énfasis en el análisis de msgstore.db de WhatsApp.

Cambio de Paradigma: Forense Clásico vs. Incident Response

Forense Clásico (Dead Box)

O1

Apagado Inmediato

Dispositivo se apaga y se coloca en Jaula de Faraday para bloquear señales.

O2

Imagen Física

Extracción bit-stream completa del almacenamiento.

O3

Problema Crítico

Cifrado FBE hace la imagen física inútil sin credenciales del usuario.

Incident Response (Live Triage)

O1

Dispositivo Activo

Se mantiene encendido para preservar artefactos volátiles en memoria.

O2

Depuración USB

Extracción lógica y de sistema de archivos mediante ADB.

O3

Ventaja Operativa

Acceso a bases de datos descifradas al vuelo y datos en memoria RAM.

- ❑ El enfoque de IR moderno prioriza la captura de datos descifrados en dispositivos activos, reconociendo que el cifrado FBE ha cambiado fundamentalmente las mejores prácticas forenses.

Toolkit Esencial para la Sesión

ADB (Android Debug Bridge)

Interfaz de línea de comandos oficial de Google para comunicación directa con el kernel de Android. Permite ejecución remota de comandos, transferencia de archivos y acceso al sistema de archivos. Es la herramienta fundamental para cualquier operación de triage en vivo.

Scrcpy

Solución de visualización de pantalla con latencia ultra-baja mediante inyección de archivos .jar en el dispositivo. Permite control remoto completo sin necesidad de acceso físico, ideal para documentación y navegación durante la investigación.

DB Browser for SQLite

Herramienta GUI para análisis profundo de estructuras de bases de datos relacionales (.db, .sqlite). Permite visualización de esquemas, ejecución de queries SQL y exportación de registros críticos para el análisis forense.

Genymotion (Entorno de Laboratorio)

Emulador Android 11 (API 30) con acceso Root preconfigurado. Proporciona un entorno controlado para practicar técnicas de extracción sin riesgo de contaminar evidencia real. Incluye Google Play Services para replicar escenarios reales.

Superficie de Ataque en Dispositivos Móviles

BYOD: El Desafío Corporativo

La política de Bring Your Own Device mezcla datos corporativos sensibles con aplicaciones y archivos personales, creando una superficie de ataque compleja donde las líneas entre seguridad empresarial y privacidad personal se difuminan constantemente.



Vectores de Ataque Principales

1

Apps Maliciosas

Instalación de APKs desde fuentes no oficiales (sideloading) que evaden los controles de seguridad de Google Play Store.

2

Permisos Abusivos

Aplicaciones que solicitan acceso a Servicios de Accesibilidad u Overlay, permitiendo captura de credenciales y keylogging.

3

Phishing via IM

Ataques dirigidos mediante WhatsApp, Telegram y otras plataformas de mensajería que explotan la confianza en contactos conocidos.

Android es Linux: Fundamentos de Arquitectura



- ❏ Diferencia crítica con Linux tradicional: Android no tiene directorios /home/user. Cada aplicación se ejecuta como un usuario distinto para el kernel, creando un modelo de seguridad basado en aislamiento por UID.

Particionamiento Estándar del Sistema

/system

Read-Only. Contiene el sistema operativo base, bibliotecas del sistema y aplicaciones preinstaladas (bloatware del fabricante). Montado como inmutable en dispositivos no rooteados.

/data (USERDATA)

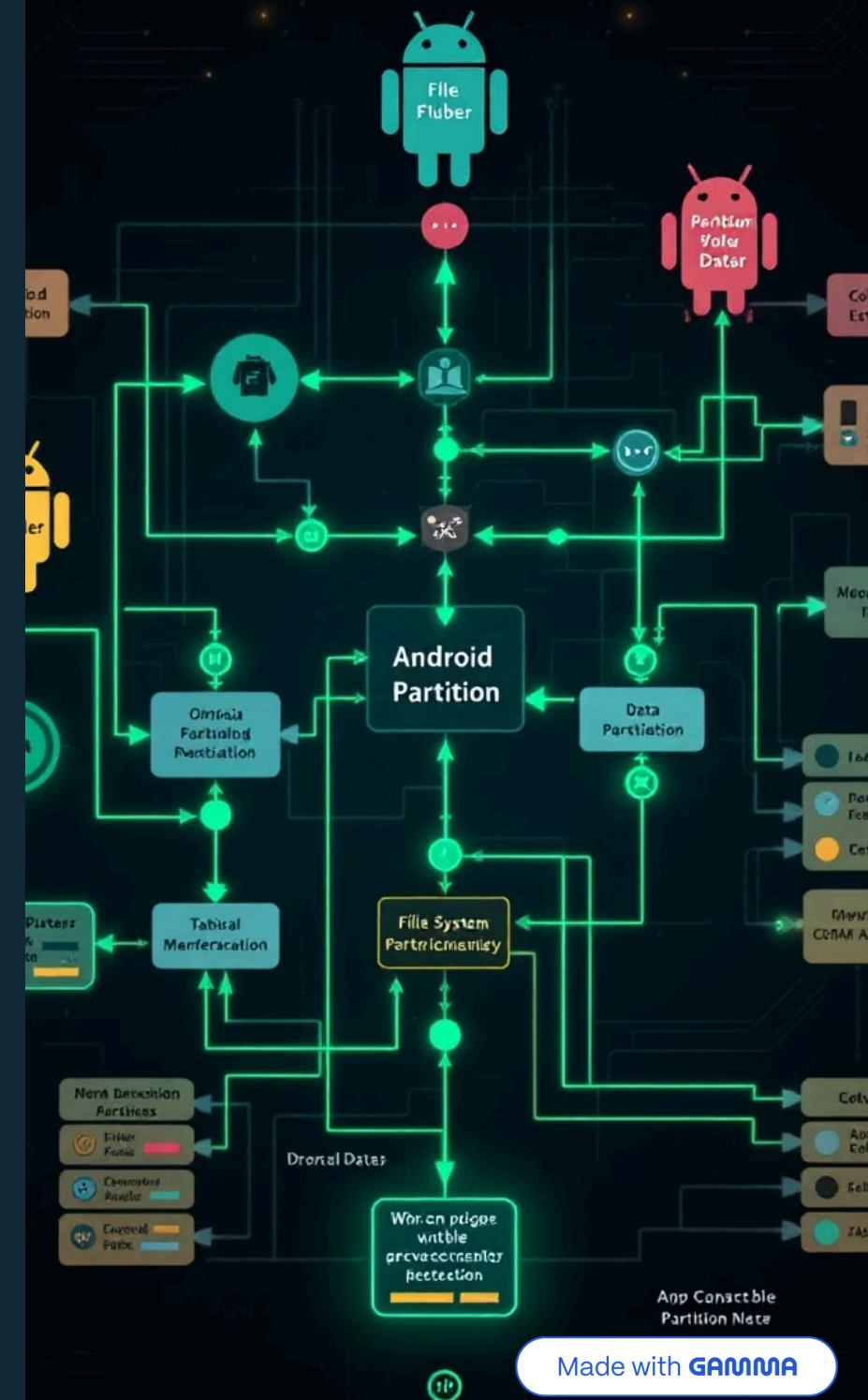
La mina de oro forense. Almacena todas las aplicaciones instaladas por el usuario, bases de datos, configuraciones y archivos privados. Completamente cifrado con FBE en dispositivos modernos.

/cache

Datos temporales del sistema, logs de actualización OTA y registros de recovery. Útil para determinar el historial de modificaciones del dispositivo.

/sdcard

Enlace simbólico a /storage/emulated/0. Almacenamiento compartido accesible por múltiples aplicaciones, sujeto a restricciones de Scoped Storage en Android 10+.



The Sandbox: Aislamiento de Aplicaciones

Modelo de Seguridad por UID

Android implementa un sistema de aislamiento único donde cada aplicación instalada recibe un User ID (UID) exclusivo al momento de la instalación. Este UID funciona como una identidad a nivel del kernel Linux.

Asignación de UID

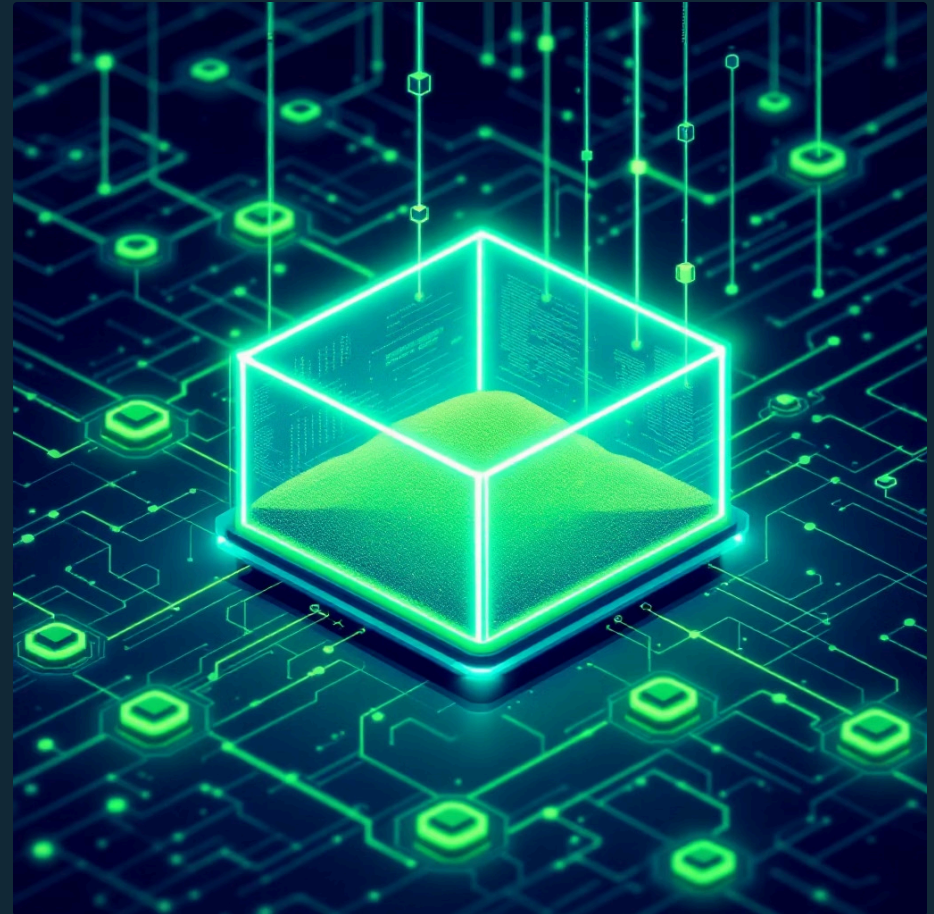
WhatsApp podría ser u0_a125, Gmail u0_a126. El prefijo u0 indica el perfil de usuario (usuario principal).

Permisos Linux Tradicionales

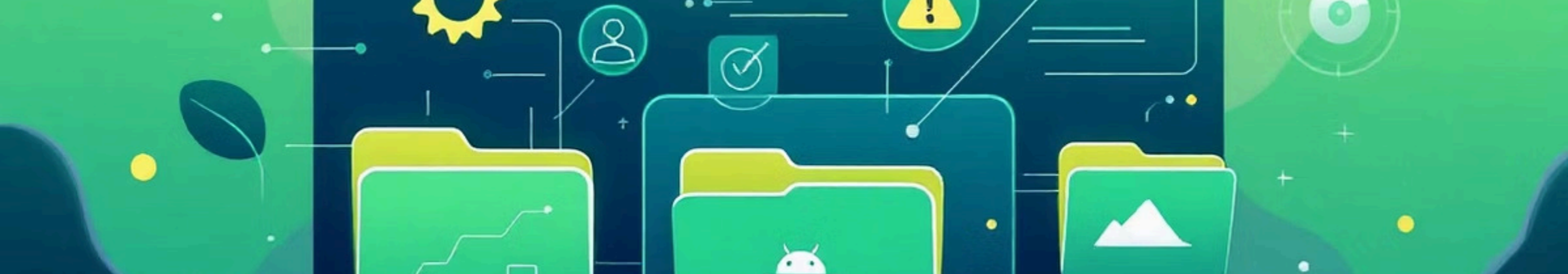
Los archivos de WhatsApp tienen permisos 700 (rwx-----) con propietario u0_a125. Gmail no puede leer estos archivos.

Acceso Privilegiado

Solo el kernel (UID 0) y procesos con privilegios Root pueden atravesar las barreras del sandbox.



❏ **Consecuencia Forense Crítica:** El comando `adb pull` ejecutado sin elevación de privilegios fallará al intentar acceder a carpetas privadas en `/data/data/`, devolviendo "Permission denied". Esto limita severamente la extracción en dispositivos no rooteados.



Scoped Storage: El Cambio de Android 10+



Legacy (Android 9-)

WhatsApp guardaba libremente en `/sdcard/WhatsApp` con acceso completo a todo el almacenamiento externo. Cualquier app con permiso `READ_EXTERNAL_STORAGE` podía leer estos datos.



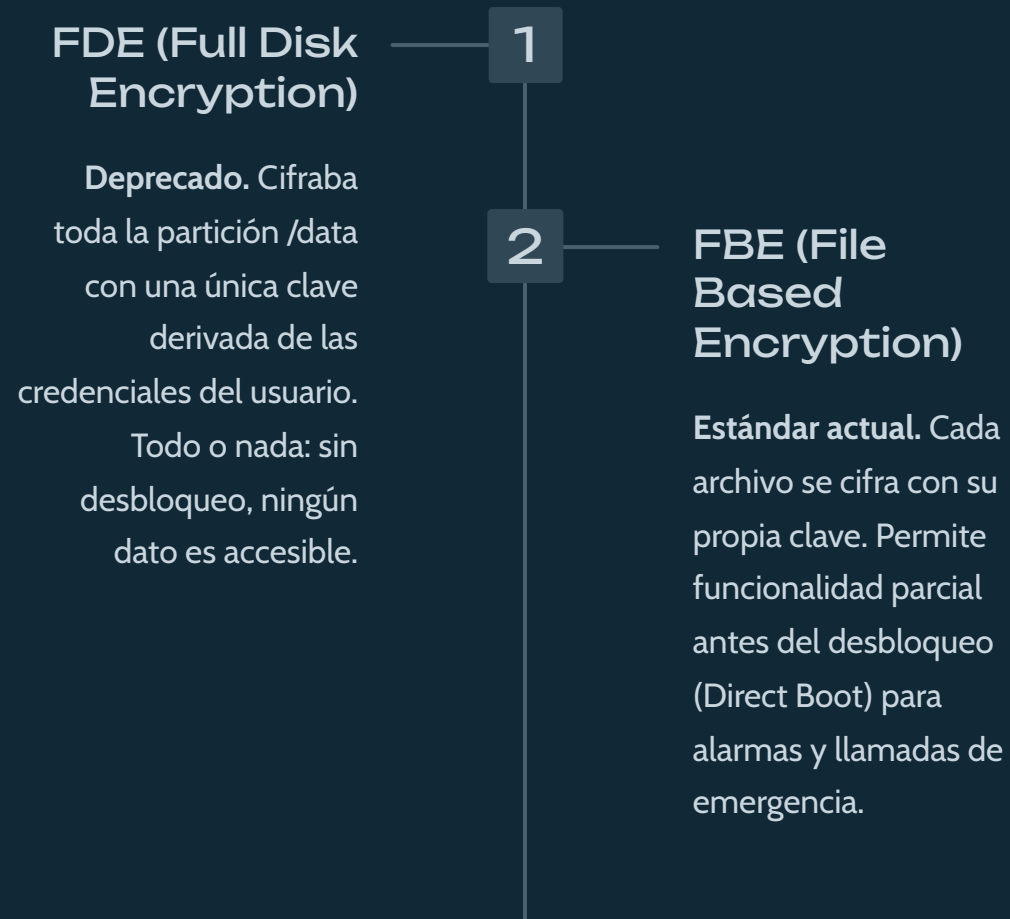
Moderno (Android 10+)

Ruta Pública: `/sdcard/Android/media/com.whatsapp/` para imágenes y videos. **Ruta Privada:** `/data/data/com.whatsapp/` para bases de datos, claves de cifrado y logs.

Este cambio radical en la arquitectura de almacenamiento fue diseñado para mejorar la privacidad del usuario, pero también complica significativamente la extracción forense. Las aplicaciones ya no pueden acceder indiscriminadamente a los archivos de otras apps, incluso con permisos de almacenamiento otorgados.

Modelo de Seguridad de Datos y Cifrado

FDE vs. FBE: Evolución del Cifrado



Impacto Forense

- ❑ Sin el PIN, patrón o contraseña del usuario, una extracción física mediante técnicas como Chip-off o JTAG devolverá únicamente datos cifrados inutilizables. El Credential Encrypted Storage permanece bloqueado hasta que se proporcionen las credenciales correctas, haciendo que las técnicas forenses invasivas tradicionales sean inefectivas contra dispositivos modernos con FBE.

256

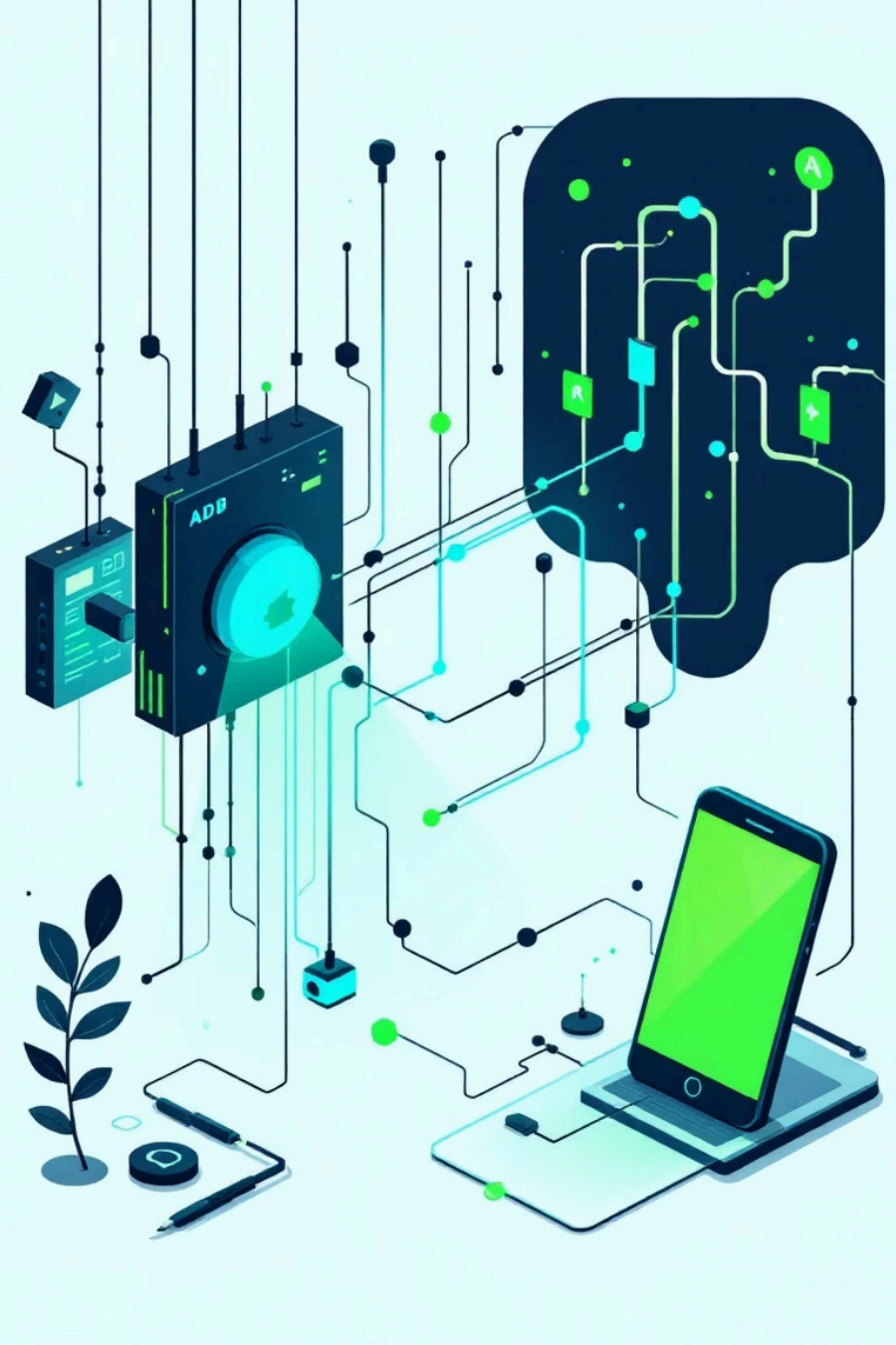
Bits de Cifrado

Algoritmo AES-256-XTS utilizado en FBE

2

Tipos de Almacenamiento

Device Encrypted (DE) y Credential Encrypted (CE)



Interacción y Protocolos de ADB en Análisis Forense Móvil

Comprendiendo la arquitectura de comunicación y los protocolos de depuración en investigaciones de respuesta a incidentes Android

El Puente de Depuración: Arquitectura ADB

ADB no es simplemente un driver USB: es un **protocolo cliente-servidor sofisticado** compuesto por tres componentes interdependientes que establecen un canal bidireccional de comunicación.



Cliente ADB

El ejecutable en tu PC forense (adb.exe). Envía comandos y recibe respuestas. Actúa como interfaz de usuario para todas las operaciones de análisis.



Servidor Local

Servicio en segundo plano en tu PC (Puerto TCP 5037). Gestiona la comunicación entre múltiples clientes y dispositivos conectados simultáneamente.



Daemon (adb)

Proceso corriendo en el dispositivo Android en espacio de usuario. Ejecuta las órdenes recibidas y retorna los resultados al servidor.

 **Seguridad del canal:** La conexión se valida mediante intercambio de llaves RSA (Whitelisting), creando un modelo de confianza que previene acceso no autorizado.

Estados de Conexión y Troubleshooting

El análisis forense falla si no comprendemos el estado del enlace entre el host y el dispositivo. Cada estado indica una condición específica que requiere acciones correctivas diferentes.



unauthorized

El daemon corre correctamente, pero la PC no tiene la llave RSA privada autorizada. **Requiere desbloqueo de pantalla** en el dispositivo y aceptación del diálogo de confianza.



offline

El dispositivo se desconectó físicamente o el daemon adbd crasheó. Verifica conexión USB, reinicia el servidor ADB con `adb kill-server` y reconecta.



device

Enlace establecido y autenticado correctamente. **Listo para recibir comandos** de análisis forense. Estado óptimo para iniciar el triaje.



recovery/sideload

Modos especiales de arranque útiles para flasheo de firmware. Funcionalidad limitada para propósitos forenses, pero permite acceso a particiones de bajo nivel.

ADB over TCP/IP: Forense Remoto

ADB puede transportar sus paquetes sobre **WiFi/Ethernet** en lugar de USB, permitiendo análisis remoto sin contacto físico directo con el dispositivo comprometido.

Comando de Activación en Laboratorio

```
# Con el dispositivo por USB, abrimos el  
puerto TCP  
adb tcpip 5555
```

```
# Desconectamos cable y conectamos  
por red  
adb connect 192.168.1.X:5555
```

Ventaja en IR: Permite triaje sin conexión física directa si estamos en la misma VPN/LAN del dispositivo objetivo.

❏ Vector de Ataque

El puerto 5555 abierto en internet es un **vector común de infección** explotado para desplegar mineros de criptomonedas y backdoors persistentes.

En investigaciones, siempre verifica si este puerto está expuesto inadvertidamente.



Reconocimiento de Objetivo (Fingerprinting)

Identificar hardware y versión del sistema operativo antes de elegir la herramienta de extracción. Esta información determina qué técnicas de adquisición son viables y qué limitaciones de seguridad enfrentaremos.

Fuente de Datos: build.prop

Archivo de configuración de arranque que almacena propiedades del sistema. Contiene información crítica para la cadena de custodia y perfilado técnico del dispositivo.

1

Modelo de Hardware

```
adb shell getprop ro.product.model
```

Ejemplo: Pixel_4, SM-G991B

2

Versión de Android

```
adb shell getprop ro.build.version.sdk
```

SDK 30 = Android 11

3

Número de Serie

```
adb shell getprop ro.serialno
```

Vital para Cadena de Custodia

Acceso a Shell y Elevación de Privilegios

Shell Estándar

Al ejecutar `adb shell`, entramos con el usuario **shell (UID 2000)**. Tiene permisos limitados: puede listar paquetes y acceder a directorios públicos, pero no puede leer datos privados de aplicaciones.

Indicador Visual

```
phonename:/ $ ← Usuario Shell
```

El símbolo `$` indica operación con privilegios restringidos.

Shell Root

Si el dispositivo está rooteado o utiliza una imagen de ingeniería, ejecutamos `su` para obtener privilegios de superusuario (UID 0).

Indicador Visual

```
phonename:/ # ← Usuario Root
```

El símbolo `#` confirma privilegios totales sobre el sistema de archivos.

❏ **Realidad en IR Corporativo:** En incidentes empresariales, casi nunca tendremos acceso Root inicial. Debemos desarrollar técnicas de análisis que operen dentro de las limitaciones del UID 2000, utilizando APIs públicas y backups lógicos.



Live Triage & Hunting en Dispositivos Android

El análisis en vivo permite capturar evidencia volátil antes de la adquisición completa. Estas técnicas identifican amenazas activas, procesos maliciosos y conexiones de comando y control en ejecución.

O1

Enumeración de Paquetes

Identificar software instalado y distinguir aplicaciones de sistema de aquellas instaladas por el usuario o atacante.

O2

Process Hunting

Detectar código en ejecución activa, malware persistente o agentes de C2 comunicándose con infraestructura externa.

O3

Análisis de Red

Identificar conexiones establecidas que puedan indicar exfiltración de datos o reverse shells activas.

O4

Revisión de Logs

Examinar eventos del sistema, crashes y actividades registradas para reconstruir la línea temporal del incidente.

Enumeración de Paquetes y Process Hunting

App Discovery: Package Manager

El **Package Manager (PM)** es la herramienta nativa para consultar la base de datos de paquetes instalados. Diferenciamos aplicaciones de sistema (inmutables, firmadas por el fabricante) de aplicaciones de usuario (sospechosas, instaladas manualmente o por malware).

```
# Listar SOLO apps de terceros (instaladas por usuario/atacante)
```

```
adb shell pm list packages -3
```

```
# Obtener la ruta física del APK para extracción posterior
```

```
adb shell pm list packages -f | grep "nombre_app"
```

```
# Output: package:/data/app/com.malware.apk=com.malware
```

Análisis de Memoria Viva: Process Status

Detectar código en ejecución activa utilizando **ps** (Process Status). Android emplea sintaxis BSD/Linux estándar para listar procesos.

USER: ¿Quién corre el proceso?

u0_a123 = Aplicación de usuario | root = Proceso del sistema con privilegios totales

PPID: Parent Process ID

¿Quién lanzó este proceso? Si una app de calculadora spawnó una shell, es comportamiento anómalo indicativo de malware.

Network Socket Analysis y Sistema Logcat

Análisis de Conexiones de Red

Objetivo: Identificar conexiones establecidas que indiquen exfiltración de datos o reverse shells activas hacia servidores de comando y control.

```
adb shell netstat -nWp | grep ESTABLISHED
```

- **-n:** No resolver DNS (ver IPs crudas sin traducción)
- **-W:** Mostrar nombres completos de procesos
- **-p:** Mostrar PID/Nombre del programa (Requiere Root en Android 10+)

Logcat: Event Viewer de Android

Sistema de **logging circular** que registra crashes, inicio de actividades y stack traces de errores. Fuente crítica de evidencia temporal.

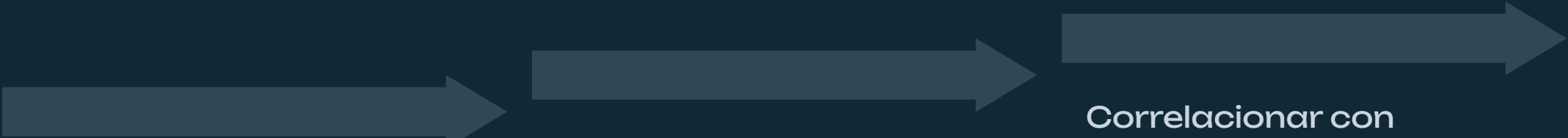
Buffers Principales

- **main:** Logs generales de aplicaciones
- **system:** Framework y servicios del SO
- **crash:** Volcados de errores fatales

```
adb logcat -d -v threadtime > triage_logs.txt
```

Detección de Persistencia vía Battery Stats

El malware sofisticado (mineros de criptomonedas, spyware) evita que el CPU entre en modo "Deep Sleep" para mantener su operación continua. Este comportamiento genera **Wakelocks** detectables que consumen batería de forma anómala.



Extraer estadísticas de batería

```
adb shell dumpsys batterystats
```

Genera un reporte detallado del consumo energético por UID y proceso en las últimas 24-48 horas.

Analizar sección "Estimated power use"

Identifica qué UIDs y procesos consumieron más energía computacional en segundo plano.

Correlacionar con enumeración de paquetes

UIDs desconocidos con **alto consumo de cómputo** en segundo plano son candidatos principales de investigación profunda.

❏ **Indicador de Compromiso:** Una aplicación legítima (ej: linterna, calculadora) que mantiene el CPU despierto durante horas es altamente sospechosa. Cruzar estos datos con análisis de red y procesos activos.